COMS3011A Lab 1

Brendan Griffiths

1 Starting a Project

Due date: 4 August 2026

1.1 Brief

Build a todo application using Next.js and SQLite.

The application is local-first. It will not be deployed to the web; a user downloads it and runs it locally via Node.js, npm or equivalent. There are no user accounts — the application serves a single user on the machine it runs on.

A full feature list:

- The user can create, edit and archive tasks.
  - Each task carries the following information: Title, Description, Due Date, Topic.
  - A task cannot be deleted, only archived, so that it remains viewable.
- The user can view their tasks as a list, sortable by topic, by status and by due date.
- Each task has one of three statuses: Todo, In-Progress, Complete. These are fixed; they are not user-customisable.
- A task that is overdue must be indicated in some way, but not as a status.
- If the application is restarted, all information persists.

Requirements:

- The repository must contain at least three tests that exercise real behaviour, runnable by a single documented command.
- The repository must contain documentation, as markdown, specifying:
  - **Third-Party Code** — the libraries and packages you installed, and one line on why each was chosen.
  - **Database Design** — the tables and the relationships between them.
  - **Running It** — the Node version and the exact commands to install, run and test the application, such that a reader can start it from a clean clone with nothing else to hand.

1.2 Submission

- The link to the GitHub repository.
- The documentation files.
- Transcripts of AI usage — planning, code generation, debugging.

1.3 Marking

Total: 100 marks. The functional walkthrough (28) is scored from the fixed checklist below. The remaining 72 marks are scored on the three-level rubric; level 1 awards half the stated weight.

1.3.1 Functional walkthrough (28 marks)

Performed from a clean clone, in this order. Each step is 4 marks, awarded pass/fail with no partial credit. Cosmetic defects are not penalised here.

1. The application installs and starts by following the README alone.
2. A task can be created carrying all four fields, and appears in the list.
3. An existing task can be edited, and the change survives a page reload.
4. A task can be archived; it leaves the active list but remains viewable.
5. The list sorts by topic, by status, and by due date.
6. A task whose due date has passed is visibly flagged, and overdue is not one of the three selectable statuses.
7. The application is stopped and restarted; all data persists.

A submission that fails step 1 is given one further attempt of at most ten minutes, after which the walkthrough scores zero and marking proceeds from the repository alone.

1.3.2 Rubric (72 marks)

Criteria	Weight	0 — Absent	1 — Partial	2 — Complete
Documentation	18	Missing, or one of the three required sections is absent.	All three sections present, but at least one is a bare list — dependencies named without justification, or a database section that does not describe the relationships — or the run instructions omit a step needed to start the application.	All three specific and accurate: each dependency with a stated reason; tables and relationships matching the shipped schema; run instructions that name the Node version and every command required, verified against a clean clone.
Commit history	18	Fewer than six commits, or a history dominated by bulk dumps — the whole application arriving in one or two commits.	Work is split into commits, but messages are largely uninformative (“fix”, “update”, “wip”), or the timestamps show the entire project committed in a single sitting.	At least six commits, each a coherent slice that leaves the repository in a working state, with messages stating what changed and why where the reason is not obvious from the diff. Work is visibly spread over more than one session.
Database design	16	No schema file or migrations; task data held in memory, in a JSON file, or otherwise not in SQLite; archived tasks deleted outright.	A schema exists and persists tasks, but at least one decision is unsound: overdue stored as a column or status value rather than derived, archive implemented by copying rows elsewhere, or the topic and status fields modelled inconsistently with the documented design.	A schema that a reader could work from: sensible column types and constraints, archive represented as a flag or timestamp on the task, overdue derived at read time from the due date and status, and the shipped schema matching what the documentation claims.
Testing	12	No tests, or tests that assert nothing — render-only smoke tests, tautological assertions.	Three or more tests exist and assert something real, but they do not run from the documented command, or they depend on the developer’s own database file and its contents.	Three or more tests exercising real behaviour, including at least one that covers archiving or the overdue rule. They are deterministic, run against a throwaway database, and pass

Criteria	Weight	0 — Absent	1 — Partial	2 — Complete
				from the single documented command.
AI usage	8	No transcripts, or transcripts showing whole-project generation with no input beyond the brief.	Task-level use with stated constraints, but no instance of the author rejecting, correcting or constraining an output.	Constraints stated up front, and at least one clear instance of the author identifying an unsuitable or incorrect output and redirecting it. Decisions visible in the transcript are traceable to the shipped code.

AI Declaration: The preceding document was reviewed and edited with: Claude-Web[Claude Opus 5]